Rapport Base de donnée

Schéma E/R : (Choix de cette modélisation + limite)

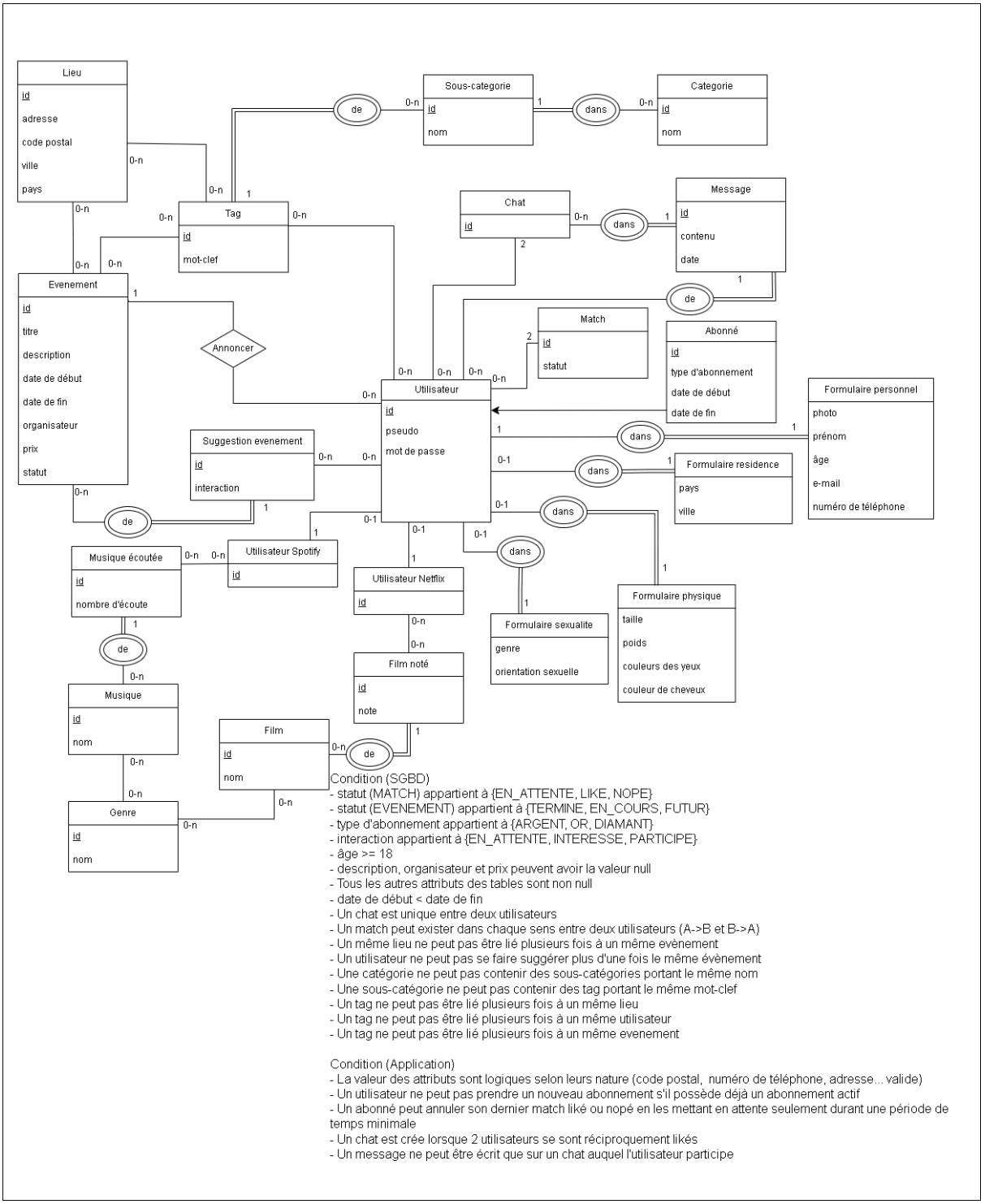


Figure 1. Schéma E/R (voir ER.png)

Schéma relationnel

utilisateur(id, pseudo, mot_de_passe)

formulaire_personnel(id_utilisateur, photo, prenom, age, email, numero_telephone)
formulaire_personnel[id_utilisateur] ⊆ utilisateur[id]

formulaire_residence(id_utilisateur, pays, ville)
formulaire_residence[id_utilisateur] ⊆ utilisateur[id]

formulaire_sexualite(id_utilisateur, genre, orientation_sexuelle)
formulaire_sexualite[id_utilisateur] ⊆ utilisateur[id]

formulaire_physique(id_utilisateur, taille, poids, couleurs_yeux, couleur_cheveux)
formulaire_physique[id_utilisateur] ⊆ utilisateur[id]

abonne(id, id_utilisateur, type_abonnement, date_debut, date_fin)
abonne[id_utilisateur] ⊆ utilisateur[id]

match(id, id_utilisateur1, id_utilisateur2, statut_match)
match[id_utilisateur1] ⊆ utilisateur[id]
match[id_utilisateur2] ⊆ utilisateur[id]

chat(id, id_utilisateur1, id_utilisateur2)
chat[id_utilisateur1] ⊆ utilisateur[id]
chat[id_utilisateur2] ⊆ utilisateur[id]

message(id, id_utilisateur, id_chat, contenu, date)
message[id_utilisateur] ⊆ utilisateur[id]
message[id_chat] ⊆ chat[id]

lieu(id, adresse, code postal, ville, pays)

evenement(id, id_utilisateur_annonceur, titre, description, date_debut, date_fin, organisateur, prix, statut_evenement)
evenement[id_utilisateur_annonceur] ⊆ utilisateur[id]

lieu_evenement(id, id_lieu, id_evenement)
lieu_evenement[id_lieu] ⊆ lieu[id]
lieu_evenement[id_evenement] ⊆ evenement[id]

suggestion_evenement(id, id_evenement, id_utilisateur, type_interaction)
suggestion_evenement[id_evenement] ⊆ evenement[id]
suggestion_evenement[id_utilisateur] ⊆ utilisateur[id]

categorie(id, nom)

sous_categorie(id, id_categorie, nom)
sous_categorie[id_categorie] ⊆ categorie[id]

tag(id, id_sous_categorie, mot_clef)
tag[id_sous_categorie] ⊆ sous_categorie[id]

tag_lieu(id, id_tag, id_lieu)
tag_lieu[id_tag] ⊆ tag[id]
tag_lieu[id_lieu] ⊆ lieu[id]

tag_utilisateur(id, id_tag, id_utilisateur)
tag_utilisateur[id_tag] ⊆ tag[id]
tag_utilisateur[id_utilisateur] ⊆ utilisateur[id]

tag_evenement(id, id_tag, id_utilisateur)
tag_evenement[id_tag] ⊆ tag[id]
tag_evenement[id_evenement] ⊆ evenement[id]

utilisateur_netflix(id_utilisateur)
utilisateur_netflix[id_utilisateur] ⊆ utilisateur[id]

film(id, nom)

film_note(id, id_film, id_utilisateur_netflix, note)
film_note[id_film] ⊆ film[id]
film_note[id_utilisateur_netflix] ⊆ utilisateur_netflix[id_utilisateur]

utilisateur_spotify(id_utilisateur)
utilisateur_spotify[id_utilisateur] ⊆ utilisateur[id]

musique(id, nom)

musique_ecoute(id, id_musique, id_utilisateur_spotify, ecoute)
musique_ecoute[id_musique] ⊆ musique[id]
musique_ecoute[id_utilisateur_spotify] ⊆ utilisateur_spotify[id_utilisateur]

genre(id, nom)

genre_film(id, id_genre, id_film)
genre_film[id_genre] ⊆ genre[id]
genre_film[id_film] ⊆ film[id]

genre_musique(id, id_genre, id_musique)
genre_musique[id_genre] ⊆ genre[id]
genre_musique[id_musique] ⊆ musique[id]

Figure 2. Schéma Relationnel (voir RELATIONNEL.png)

Commentaires par rapport à la structure et modélisation de la base de données

Idée de base :
Pour construire notre base de données, nous nous sommes imaginé à quoi ressemblerait un site internet de celui-ci et quelles données devraient ou non y être stockées. Un utilisateur en soit, contient déjà beaucoup d'informations notamment lors de l'inscription. Il remplit son identité et des tables appelées "Formulaires", notamment pour éviter d'encombrer la table "Utilisateur".

Match, chat et messages :
Pour les "Match", nous sommes partis du principe que les utilisateurs ont tous des goûts très variés et qu'il ne serait pas impossible que 2 utilisateurs aient tellement de goûts opposés, qu'ils ne soient jamais suggéré l'un ou l'autre (les contraires ne s'attirent pas dans notre cas). Un tuple match est composé de l'id de 2 utilisateurs, la source et la cible. L'existence de ce tuple indique qu' à travers l'algorithme de matching permet à 2 utilisateurs de potentiellement trouver l'amour ! Chaque tuple contient également une information concernant l'étape de l'utilisateur source par rapport à la cible : "EN_ATTENTE", cela indique que l'utilisateur cible est compris dans la "liste" de l'utilisateur source et qu'il pourra "LIKE" ou "NOPE" à travers l'application. L'existence de ce tuple entraîne également l'existence du tuple réciproque (A->B) et (B->A).
Lorsque les 2 utilisateurs se sont "LIKE", un chat est créé où ils pourront discuter.

Evènements et tags :
Une section évènement est également disponible sur l'application.
Les utilisateurs peuvent annoncer des évènements et montrer aux autres utilisateurs s'ils ont l'intention de participer ou qu'ils sont simplement intéressés. Les utilisateurs pourront aussi ajouter des "tags" qui agissent comme des mot-clé ou thème sur les événements et directement sur son profil. Les utilisateurs qui "jouent le jeu" le plus permet à l'algorithme de l'application de mieux faire ses choix. Des événements peuvent ainsi être suggérés à l'utilisateur à travers son fil personnalisé et autres moyens (notifications, publicités, …)

Applications tierces :
L'utilisateur peut lier des comptes d'applications tierces à son compte Big Match comme par exemple Spotify ou Netflix. Elle pourra donc récupérer les données déjà émises sur ces sites. Pour Spotify, il serait possible d'avoir le nombre d'écoutes des musiques qu'il écoute et sur Netflix, les notes sur les films qu'il a vu. La table

auxiliaire “Genre” agit comme un “tag” pour les tables “Film” et “Musique” et permet d’en savoir plus sur ses préférences culturelles.

Evolution et limites :
Nous sommes partis sur une structure qui nous semble claire et assez extensible pour des améliorations futures. Chaque table joue un rôle bien défini dans le système sans pour autant complexifier l’ensemble. Cependant, le système actuel est très dépendant de ses utilisateurs concernant les données du système. Des utilisateurs peu investis ou un manque peut fragiliser l’algorithme de suggestion le rendant obsolète. Il faut également prendre en compte les pratiques éthiques de ceux qui utilisent cette application poussant à limiter le stockage des informations personnelles des utilisateurs.

Requêtes :

Requête 1 :

Condition : Requête qui porte sur au moins trois tables
Description : Utilisateurs abonnés ayant au moins un match

Requête 2 :

Condition : Auto jointure ou Jointure réflexive
Description : Pseudo des utilisateurs qui se sont likés mutuellement

Requête 3 :

Condition : Sous-requête corrélée
Description : Utilisateurs qui n'ont pas encore répondu à une suggestion d'événement (en attent)

Requête 4 :

Condition : Sous-requête dans le FROM
Description : Requête qui récupère tous les évènements qui ont été assignés au moins un lieu dont le pays est la France. Parmi ceux-là, on récupère seulement les événements qui n'ont pas encore commencé.

Requête 5 :

Condition : Sous-requête dans le WHERE
Description : Utilisateurs qui a au moins un tag de la sous-catégorie Sports

Requête 6 :

Condition : Deux agrégats nécessitant GROUP BY et HAVING
Description : Les utilisateurs likés par 10% du nombre d'utilisateurs total.

Requête 7 :

Condition : Requête impliquant le calcul de deux agrégats
Description : Total d'écoutes et maximum d'écoutes des utilisateurs qui ont au moins écouter deux musiques sur Spotify

Requête 8 :

Condition : Jointure externe (LEFT JOIN, RIGHT JOIN ou FULL JOIN)
Description : Donne la liste des utilisateurs ainsi que leurs abonnements

Requête 9 :

Condition : 1. Requête équivalente exprimant une condition de totalité avec sous requêtes corrélées
Description : Utilisateurs qui s'est INTERESSER et PARTICIPER à tous les événements

Requête 10 :

Condition : 2. Requête équivalente exprimant une condition de totalité avec agrégation
Description : Utilisateurs qui s'est INTERESSER et PARTICIPER à tous les événements

Requête 11 :

Condition : 1. Requête non équivalente à cause de null
Description : Évènements dont l'organisateur n'est pas Subway ou n'est pas défini

Requête 12 :

Condition : 2. Requête non équivalente à cause de null
Description : Évènements dont l'organisateur n'est pas Subway ou n'est pas défini

Requête 13 :

Condition : 1bis. Requête équivalente avec information incomplète
Description : Évènements dont l'organisateur n'est pas Subway

Requête 14 :

Condition : 2bis. Requête équivalente avec information incomplète
Description : Évènements dont l'organisateur est défini et n'est pas Subway

Requête 15 :

Condition : Requête récursive
Description : Tous les messages envoyés dans le chat 1 dans l'ordre chronologique

Requête 16 :

Condition : Requête utilisant du fenêtrage
Description : Renvoie les 3 derniers messages de chaque utilisateur

Algorithme de matching : (Choix pour les calculs de recommandation)

Requêtes possibles pour l’algo de matching:

- Genre & Orientation
- Tags utilisateur
- Ville & Pays
- Âge
- Historique événement
- Historique match
- Photo (Selon les nope et like)
- Taille (Selon les nope et like)
- Poids (Selon les nope et like)
- Couleurs des yeux (Selon les nope et like)
- Couleurs de cheveux (Selon les nope et like)
- Traces numériques (Spotify et Netflix)

Requête sur l'orientation sexuelle et le genre :

```
SELECT
        u1.id_utilisateur,
        u2.id_utilisateur AS utilisateur_cible
FROM formulaire_sexualite u1
JOIN formulaire_sexualite u2 ON u1.id_utilisateur != u2.id_utilisateur
WHERE(
        u1.genre = 'Femme' AND u1.orientation_sexuelle = 'Hétérosexuel'
        AND
        u2.genre = 'Homme' AND u2.orientation_sexuelle = 'Hétérosexuel'
) OR (
        u1.genre = 'Homme' AND u1.orientation_sexuelle = 'Hétérosexuel'
        AND
        u2.genre = 'Femme' AND u2.orientation_sexuelle = 'Hétérosexuel'
);
```

Requête sur les tags utilisateurs - utilisateurs:

```
SELECT
        u1.id_utilisateur,
        u2.id_utilisateur AS utilisateur_compatible,
        COUNT(*) AS nb_tags_communs
FROM tag_utilisateur u1
JOIN tag_utilisateur u2 ON u1.id_tag = u2.id_tag AND u1.id_utilisateur != u2.id_utilisateur
GROUP BY u1.id_utilisateur, u2.id_utilisateur;
```

Requête sur les tags evenements - utilisateurs:

```
SELECT
        u.id AS utilisateur_id,
        e.id AS evenement_id,
        COUNT(te.id_tag) AS nb_tags_communs
FROM utilisateur u
JOIN tag_utilisateur tu ON u.id = tu.id_utilisateur
JOIN tag_evenement te ON tu.id_tag = te.id_tag
JOIN evenement e ON te.id_evenement = e.id
```

```
WHERE e.statut_evenement = 'FUTUR'
GROUP BY u.id, e.id;
```

Requête sur la résidence des utilisateurs :

```
SELECT
    u1.id AS utilisateur_id,
    u2.id AS utilisateur_compatible_id
FROM utilisateur u1
JOIN formulaire_residence fr1 ON u1.id = fr1.id_utilisateur
JOIN formulaire_residence fr2 ON fr1.ville = fr2.ville AND fr1.pays = fr2.pays
JOIN utilisateur u2 ON fr2.id_utilisateur = u2.id
WHERE u1.id != u2.id;
```

Requête sur les lieux des événements & résidence utilisateurs :

```
SELECT
    u.id AS utilisateur_id,
    e.id AS evenement_id,
    e.titre
FROM utilisateur u
JOIN formulaire_residence fr ON u.id = fr.id_utilisateur
JOIN lieu_evenement le ON le.id_lieu IN (
    SELECT id FROM lieu WHERE ville = fr.ville AND pays = fr.pays
)
JOIN evenement e ON le.id_evenement = e.id
JOIN lieu l ON le.id_lieu = l.id
WHERE e.statut_evenement = 'FUTUR';
```

Calcul de l'indice de recommandation :

Pour les utilisateurs :
Total (en %) = requête genre & orientation + requête tags communs utilisateurs + requête résidence + autre requête

Pour les événements :
Total (en %) = requête tags communs + requête lieux événements & résidence utilisateurs + autre requête

Les utilisateurs et événements ont un calcul différent car le but des requêtes le sont également. Pour le cas des utilisateurs, on pourra établir un % d'affinité (indice de recommandation) sur les genres et l'orientation, les tags en communs et leurs lieux de résidence. Toutes ces requêtes auront un certain pourcentage sur le total, et ainsi plus on a de choses en commun plus on est compatible.

Toutefois, ce calcul est loin d'être complet et parfait, par exemple sur les genres et orientation, celui ci ne fonctionne que pour les hommes et femmes hétéro, alors qu'il devrait fonctionner pour tous les genres et orientation sexuelles. Les tags communs peuvent être aussi trompeurs, dans le cas où une personne cherche un opposé et non un commun. Les

lieux de résidence rapportent des points uniquement si on est dans la même ville alors qu'un système de distance serait meilleur. Et enfin, ce n'est qu'une partie des requêtes possibles dans l'algo de matching, il est donc pas assez fiable et complet.
Toutefois, si on remplit toutes les conditions de cet algo de matching, dans ce cas cela peut être très bénéfique pour l'utilisateur.
Afin de l'améliorer, l'idéal serait de le rendre d'avantage complet, ouvert et accessible à tous et notamment de faire des requêtes plus précises (selon leurs LIKE et NOPE) ou encore leurs traces numériques.